

0.1 Tập dữ liệu

Bộ dữ liệu được sử dụng trong đề tài là Jigsaw Toxic Comment Classification Challenge từ Kaggle **kaggleJigsaw** với mục tiêu phát triển các mô hình có khả năng nhận diện các loại bình luận độc hại khác nhau trên Wikipedia. Bộ dữ liệu này bao gồm các bình luận từ Wikipedia đã được gán nhãn thủ công bởi các chuyên gia kiểm duyệt theo 6 loại độc hại khác nhau. Tổng cộng có 159,571 bình luận trong tập dữ liệu, được chia theo tỷ lệ 80% cho tập huấn luyện và 20% cho tập kiểm định, sử dụng phương pháp chia tách phân tầng để đảm bảo phân phối các lớp được giữ nguyên trong cả hai tập.

Phân tích chi tiết phân bố nhãn cho thấy mức độ mất cân bằng nghiêm trọng của dữ liệu. Lớp toxic có khoảng 15,000 mẫu dương tính (chiếm khoảng 9.4% tổng số mẫu), lớp obscene có khoảng 8,000 mẫu (5.0%), và lớp insult có khoảng 7,600 mẫu (4.8%). Các lớp thiểu số đặc biệt nghiêm trọng: severe_toxic chỉ có khoảng 1,600 mẫu (1.0%), identity_hate có khoảng 1,400 mẫu (0.9%), và đặc biệt threat chỉ có khoảng 320 mẫu (0.2%) - một con số rất nhỏ so với tổng thể 159,571 bình luận. Sự mất cân bằng này đặt ra thách thức lớn cho việc huấn luyện mô hình, buộc phải áp dụng các kỹ thuật xử lý mất cân bằng như Class-Weighted Loss để đảm bảo mô hình không bỏ qua hoàn toàn các lớp thiểu số. Hiện tượng này cũng đã được nghiên cứu trong nhiều bài toán phân loại văn bản khác, như đã được báo cáo bởi các nghiên cứu trước đây về phát hiện lời lẽ thù hận **djuric2015hate**.

Tập kiểm định chứa 31,915 bình luận, được sử dụng để đánh giá hiệu suất của mô hình và thực hiện các thử nghiệm như tìm kiếm các ngưỡng tối ưu. Tập kiểm thử chứa khoảng 153,000 bình luận không có nhãn, được sử dụng để tạo bài nộp cho cuộc thi Kaggle. Việc đánh giá trên tập kiểm định thay vì tập kiểm thử (không có nhãn thực) là cách tiếp cận tiêu chuẩn trong nghiên cứu, cho phép so sánh và phân tích chi tiết hiệu suất của từng mô hình. Raschka (2018) đã thảo luận chi tiết về tầm quan trọng của việc sử dụng tập kiểm định để đánh giá và lựa chọn mô hình, tránh để tập kiểm thử ảnh hưởng đến quá trình phát triển mô hình **raschka2018roc**.

Bên cạnh Jigsaw, đề tài sử dụng bộ dữ liệu HateXplain **hatexplain2021** để đánh giá chéo. HateXplain **hatexplain2021** bao gồm 20,109 bình luận được thu thập từ Twitter và Reddit, được gán nhãn bởi ba người gán nhãn độc lập với ba nhãn: normal (bình thường), offensive (xúc phạm), và hatespeech (lời thù hận). Khác với Jigsaw (đa nhãn, 6 nhãn độc lập), HateXplain là bài toán đa lớp với một nhãn duy nhất cho mỗi bình luận. Dữ liệu được chia theo tỷ lệ 80% cho huấn luyện (16,087 mẫu) và 20% cho kiểm định (4,022 mẫu). Việc sử dụng hai bộ dữ liệu với nguồn gốc khác nhau (Wikipedia so với Twitter/Reddit) và cấu trúc nhãn khác nhau (đa

nhân so với đa lớp) cho phép đánh giá tính tổng quát của các kiến trúc mô hình trên nhiều điều kiện dữ liệu.

0.2 Cấu hình thử nghiệm

Để đảm bảo tính công bằng trong việc so sánh giữa các mô hình, tất cả các mô hình đều được huấn luyện với cùng bộ siêu tham số và chỉ khác nhau về kiến trúc mô hình. Các siêu tham số chính bao gồm: kích thước batch = 64, tốc độ học = 2×10^{-3} , số epoch tối đa = 10 với Early Stopping (patience = 2), độ dài tối đa chuỗi = 128 token, chiều embedding = 128, chiều hidden state = 256, số tầng RNN = 2, và tỷ lệ dropout = 0.3. Ngoài ra, các tham số tối ưu hóa bao gồm weight decay = 10^{-4} , tỷ lệ warmup = 0.1, và giới hạn gradient clip = 1.0. Việc sử dụng chung siêu tham số cho tất cả các mô hình là một phương pháp phổ biến trong nghiên cứu so sánh các kiến trúc mô hình, như được áp dụng trong nhiều nghiên cứu trước đây [vaswani2017transformer](#), [lai2015rcnn](#).

Tokenizer được sử dụng là AutoTokenizer từ thư viện Hugging Face Transformers [wolf2020transformers](#) với mô hình đã được huấn luyện trước "distilbert-base-uncased". DistilBERT là phiên bản nhỏ gọn hơn và nhanh hơn BERT, được huấn luyện trên 6GB dữ liệu văn bản tiếng Anh, do đó tokenizer của nó có khả năng tách từ thông minh, hiểu được subword tokenization (tách từ thành các tiền tố và hậu tố) giúp xử lý tốt các từ hiếm gặp và từ không có trong từ điển. Sanh và cộng sự (2019) đã chứng minh rằng DistilBERT đạt 97% hiệu suất của BERT trên bộ đánh giá GLUE trong khi chỉ sử dụng 60% tham số và thời gian suy luận nhanh hơn 60% [sanh2019distilbert](#).

Cấu hình chi tiết cho từng mô hình như sau: LSTM và Attention LSTM sử dụng 2 tầng LSTM với hidden size 256, embedding dimension 128; BiLSTM và Attention BiLSTM sử dụng 2 tầng LSTM forward và 2 tầng LSTM backward với hidden size 256 mỗi hướng; GRU sử dụng 2 tầng GRU với hidden size 256; RCNN sử dụng 2 tầng BiLSTM với hidden size 256 mỗi hướng, tầng fusion với hàm kích hoạt tanh; Transformer sử dụng 2 tầng mã hóa, 8 đầu attention, feed-forward dimension 512. Tất cả các mô hình đều sử dụng dropout 0.3 trước lớp fully connected cuối cùng.

Song song với thử nghiệm gốc, đề tài tiến hành thử nghiệm bổ sung với cấu hình tối ưu hóa trên cả hai bộ dữ liệu. Cấu hình mới sử dụng batch size = 32, tốc độ học = 1×10^{-3} , gradient accumulation với 2 bước (batch size hiệu quả = 64), số epoch tối đa = 5, và OneCycleLR scheduler. Về kiến trúc, Transformer được nâng cấp lên 4 tầng mã hóa với Pre-LN và CLS token thay vì Post-LN và mean pooling, RCNN được mở rộng thêm một lớp fc_hidden với hàm kích hoạt ReLU sau max pooling (EnhancedRCNN), và Attention BiLSTM giữ nguyên kiến trúc self-attention gốc.

Masked pooling, label smoothing ($\epsilon = 0.1$), và focal loss ($\gamma = 2.0$) cũng được áp dụng.

0.3 So sánh tổng quan các mô hình

Kết quả huấn luyện và đánh giá trên tập kiểm định cho thấy sự khác biệt đáng kể giữa các mô hình. Bảng tổng hợp dưới đây trình bày các chỉ số hiệu suất chính của tất cả 7 mô hình đã được thử nghiệm. Các độ đo được sử dụng bao gồm ROC-AUC (diện tích dưới đường cong ROC), F1 Macro (trung bình F1 của các lớp), và F1 Micro (F1 tính tổng hợp trên tất cả các mẫu và nhãn), được tính toán dựa trên các công thức đã được phân tích chi tiết bởi Sokolova và Lapalme (2009) trong nghiên cứu về các độ đo cho bài toán phân loại **sokolova2009systematic**.

Mô hình	Tham số (M)	Best AUC	Final AUC	F1 Macro	F1 Micro
RCNN	6.6	0.9839	0.9831	0.5689	0.6655
Attention BiLSTM	6.5	0.9834	0.9830	0.6311	0.7324
Attention LSTM	4.9	0.9812	0.9791	0.5650	0.6820
GRU	4.6	0.9805	0.9793	0.5146	0.6198
BiLSTM	6.3	0.9800	0.9776	0.5520	0.6689
Transformer	4.3	0.9789	0.9788	0.4625	0.5947
LSTM	4.8	0.9786	0.9761	0.5459	0.6658

Bảng 1: So sánh hiệu suất các mô hình trên tập kiểm định (159,571 mẫu)

Phân tích kết quả cho thấy RCNN đạt ROC-AUC cao nhất (0.9839), cho thấy khả năng xếp hạng các mẫu dương tính cao hơn các mẫu âm tính của mô hình này là tốt nhất. Attention BiLSTM đứng thứ hai với AUC 0.9834 nhưng đạt F1 Macro cao nhất (0.6311), cho thấy sự cân bằng tốt nhất giữa precision và recall. Sự khác biệt về độ đo này là do AUC đo khả năng phân biệt ở mọi ngưỡng, trong khi F1 Macro tập trung vào hiệu suất tại các ngưỡng tối ưu cho từng lớp. Fawcett (2006) đã giải thích chi tiết sự khác biệt giữa AUC và các độ đo phụ thuộc ngưỡng như F1-score **fawcett2006introduction**.

Các mô hình không có attention (BiLSTM, LSTM, GRU) và Transformer đều có F1 Macro thấp hơn đáng kể so với các mô hình có attention (Attention BiLSTM, Attention LSTM). Cụ thể, Attention BiLSTM (0.6311) cao hơn BiLSTM (0.5520) khoảng 0.08 điểm, và Attention LSTM (0.5650) cao hơn LSTM (0.5459) khoảng 0.02 điểm. Điều này chứng minh rằng cơ chế attention thực sự mang lại lợi ích đáng kể trong bài toán phân loại bình luận độc hại, cho phép mô hình tập trung vào các từ quan trọng và cải thiện khả năng phân biệt giữa các trường hợp thực sự thuộc nhãn và các trường hợp dương tính giả. Kết quả này nhất quán với các nghiên cứu trước đây về hiệu quả của attention trong các bài toán NLP **bahdanau2014attention**, **yang2016hierarchical**.

Đáng chú ý, GRU với ít tham số nhất (4.6 triệu) và huấn luyện nhanh hơn các mô hình BiLSTM đạt F1 Macro thấp nhất trong nhóm RNN (0.5146), cho thấy thiết kế đơn giản hóa của GRU có thể không đủ để nắm bắt các phụ thuộc phức tạp trong bài toán này. Transformer với kiến trúc hoàn toàn khác (không có tính hồi quy) đạt F1 Macro thấp nhất (0.4625), có thể do cần nhiều dữ liệu và thời gian huấn luyện hơn để đạt hiệu suất tối ưu. Kết quả này cho thấy rằng kiến trúc phù hợp cho một bài toán cụ thể cần được lựa chọn dựa trên đặc điểm của bài toán và dữ liệu, như đã được nhấn mạnh trong nghiên cứu về "No Free Lunch Theorems" của Wolpert (1997) **wolpert1997no**.

0.4 Kết quả chi tiết từng mô hình

0.4.1 Kết quả của RCNN

RCNN đạt ROC-AUC cao nhất trong tất cả các mô hình (0.9839) và chi tiết hiệu suất trên từng lớp cho thấy mô hình này đặc biệt xuất sắc trong việc nhận diện bình luận obscene với F1-score 0.81 (precision 0.76, recall 0.87) và bình luận threat với F1-score 0.43 (precision 0.29, recall 0.79). Điểm mạnh của RCNN nằm ở kiến trúc kết hợp ngữ cảnh hai chiều từ BiLSTM với embedding gốc, tạo ra biểu diễn phong phú cho mỗi từ, sau đó sử dụng max pooling để trích xuất các đặc trưng nổi bật nhất. Lai và cộng sự (2015) đã đề xuất kiến trúc RCNN trong bài báo tại AAAI và chứng minh hiệu quả của nó trên nhiều bài toán phân loại văn bản **lai2015rcnn**. Cách tiếp cận này đặc biệt hiệu quả khi các từ độc hại (như "fuck", "shit" cho obscene) xuất hiện rõ ràng và max pooling có thể dễ dàng chọn ra chúng.

Tuy nhiên, RCNN cũng có hạn chế rõ rệt trên các lớp thiểu số: severe_toxic chỉ đạt precision 0.25 với recall 0.96 ($F1 = 0.40$), và identity_hate chỉ đạt precision 0.20 với recall 0.85 ($F1 = 0.32$). Precision thấp trên các lớp này cho thấy mô hình có xu hướng dự đoán quá nhiều, tạo ra nhiều dương tính giả. Nguyên nhân có thể là do max pooling chỉ chọn giá trị lớn nhất, có thể bỏ qua các tín hiệu quan trọng trong các trường hợp không có từ khóa rõ ràng. Khi áp dụng các ngưỡng tối ưu, F1 Macro tăng lên 0.6328 và F1 Micro tăng lên 0.7428, cho thấy việc tinh chỉnh ngưỡng vẫn mang lại cải thiện đáng kể.

Một điểm cần lưu ý là RCNN có số lượng tham số nhiều nhất (6.6 triệu) trong số tất cả các mô hình, đòi hỏi nhiều bộ nhớ GPU và thời gian huấn luyện lâu hơn. Việc sử dụng BiLSTM 2 tầng để tạo ngữ cảnh hai chiều cộng với tầng fusion tạo ra chi phí tính toán cao hơn so với các mô hình khác. Tuy nhiên, với ROC-AUC cao nhất, RCNN vẫn là một lựa chọn tốt cho các ứng dụng cần ưu tiên khả năng phân biệt tổng thể.

0.4.2 Kết quả của Attention BiLSTM

Attention BiLSTM đạt F1 Macro cao nhất (0.6311 với các ngưỡng tối ưu = 0.6503) và ROC-AUC thứ hai (0.9834), cho thấy đây là mô hình có sự cân bằng tốt nhất giữa các độ đo hiệu suất. Kết quả chi tiết cho thấy mô hình đạt được precision cao hơn đáng kể so với RCNN ở hầu hết các lớp, đặc biệt là toxic (0.67 so với 0.62), severe_toxic (0.29 so với 0.25), và identity_hate (0.32 so với 0.20). Đồng thời, Attention BiLSTM vẫn duy trì recall ở mức tương đối cao cho hầu hết các lớp, với recall cao nhất ở obscene (0.94) và insult (0.92). Sự kết hợp giữa BiLSTM và attention đã được nghiên cứu trước đó bởi Yang và cộng sự (2016) trong Hierarchical Attention Networks và đã chứng minh hiệu quả trên nhiều bài toán phân loại văn bản **yang2016hierarchical**.

Điểm nổi bật của Attention BiLSTM là khả năng đạt F1 cao nhất trên lớp threat (0.48 với các ngưỡng tối ưu) so với tất cả các mô hình khác. Điều này cho thấy cơ chế attention giúp mô hình học được cách nhận diện các mẫu đe dọa ngay cả khi chúng không chứa từ khóa rõ ràng, bằng cách tổng hợp thông tin từ nhiều phần của câu. Tuy nhiên, recall cho threat vẫn chỉ ở mức 0.68, thấp hơn nhiều so với các lớp khác, cho thấy vẫn còn khó khăn trong việc nhận diện tất cả các bình luận threat trong tập dữ liệu. Nguyên nhân có thể là do số lượng mẫu threat quá ít (chỉ 320 mẫu), khiến mô hình khó học đầy đủ các biểu diễn cần thiết.

Số lượng tham số của Attention BiLSTM là 6.5 triệu, gần tương đương với RCNN (6.6 triệu), và thời gian huấn luyện cũng tương tự (khoảng 9 phút mỗi epoch). Tuy nhiên, với F1 Macro cao hơn và precision tốt hơn trên các lớp thiểu số, Attention BiLSTM có thể được coi là lựa chọn tốt hơn cho các ứng dụng cần cân bằng precision và recall.

0.4.3 Kết quả của BiLSTM

BiLSTM không có attention đạt ROC-AUC 0.9800 và F1 Macro 0.5520 (với các ngưỡng tối ưu = 0.5825), thấp hơn đáng kể so với Attention BiLSTM. Kết quả này xác nhận vai trò quan trọng của cơ chế attention trong việc cải thiện hiệu suất phân loại. BiLSTM sử dụng Global Average Pooling thay vì attention pooling, nghĩa là tất cả các bước thời gian được tính bình quân với trọng số bằng nhau, khiến các từ ít quan trọng (như "the", "a", "is") cũng đóng góp ngang bằng với các từ quan trọng trong biểu diễn cuối cùng. Graves và cộng sự (2012) đã nghiên cứu về việc sử dụng RNN hai chiều trong nhận dạng giọng nói và nhận thấy rằng việc xử lý hai chiều mang lại cải thiện đáng kể so với RNN một chiều **graves2012supervised**.

Mặc dù recall trung bình của BiLSTM khá cao (0.88), precision lại rất thấp (0.36), dẫn đến F1 Macro thấp. Điều này cho thấy BiLSTM có xu hướng dự đoán

quá nhiều các trường hợp là độc hại, tạo ra nhiều dương tính giả. Cụ thể, trên lớp threat BiLSTM chỉ đạt precision 0.13, thấp hơn rất nhiều so với Attention BiLSTM (0.37), cho thấy việc thiếu cơ chế attention khiến mô hình không thể phân biệt tốt giữa các bình luận thực sự là threat và các bình luận có từ ngữ tương tự nhưng không mang ý nghĩa đe dọa. BiLSTM có 6.3 triệu tham số và huấn luyện nhanh hơn một chút so với Attention BiLSTM (do không có tầng attention), nhưng khoảng cách về hiệu suất là đáng kể.

0.4.4 Kết quả của Transformer

Transformer đạt ROC-AUC 0.9789 nhưng F1 Macro thấp nhất trong tất cả các mô hình (0.4625 với các ngưỡng tối ưu = 0.5593). Đây là kết quả đáng chú ý vì Transformer là kiến trúc hiện đại và mạnh mẽ nhất trong danh sách, được sử dụng rộng rãi trong các mô hình ngôn ngữ lớn. Vaswani và cộng sự (2017) đã giới thiệu Transformer trong bài báo "Attention Is All You Need" và chứng minh hiệu quả vượt trội trên bài toán dịch máy **vaswani2017transformer**. Tuy nhiên, kết quả trong đề tài này cho thấy Transformer cần được điều chỉnh cấu hình phù hợp để đạt hiệu suất tối ưu trên bài toán phân loại bình luận độc hại.

Nguyên nhân chính của hiệu suất thấp có thể bao gồm: thứ nhất, việc sử dụng embedding khởi tạo ngẫu nhiên thay vì các embedding đã được huấn luyện trước như GloVe **pennington2014glove** hoặc FastText **mikolov2013word2vec** khiến mô hình phải học tất cả các biểu diễn từ đầu, đòi hỏi nhiều dữ liệu và thời gian hơn; thứ hai, Transformer thường cần huấn luyện nhiều epoch hơn (20-50 epoch) để đạt hiệu suất tối ưu, trong khi thử nghiệm chỉ chạy 10 epoch; thứ ba, cấu hình 2 tầng và 8 đầu attention có thể chưa đủ mạnh để nắm bắt các phụ thuộc phức tạp trong bài toán này. Devlin và cộng sự (2019) đã chứng minh rằng việc sử dụng BERT đã được huấn luyện trước với tinh chỉnh mang lại cải thiện vượt bậc trên nhiều bài toán NLP **devlin2019bert**.

Transformer có ưu điểm về tốc độ huấn luyện: mỗi epoch chỉ mất khoảng 2 phút (nhanh hơn 4-5 lần so với BiLSTM) do xử lý song song toàn bộ chuỗi thay vì tuần tự. Số lượng tham số cũng ít nhất (4.3 triệu). Nếu được huấn luyện với các embedding đã được huấn luyện trước và nhiều epoch hơn, Transformer có tiềm năng đạt hiệu suất cao hơn.

0.5 Phân tích ảnh hưởng của các kỹ thuật tối ưu hóa

Các kỹ thuật tối ưu hóa đã được áp dụng trong đề tài đã đóng góp đáng kể vào việc cải thiện hiệu suất của tất cả các mô hình. So sánh với kết quả từ các thử nghiệm trước đó trên tập 10,000 mẫu, việc huấn luyện trên toàn bộ tập dữ liệu (159,571 mẫu) mang lại cải thiện vượt trội: ROC-AUC tăng từ khoảng 0.95 lên 0.98

(tăng khoảng 0.03 điểm), và F1 Macro tăng từ khoảng 0.43 lên 0.63 (tăng khoảng 0.20 điểm). Điều này chứng minh tầm quan trọng của việc huấn luyện trên dữ liệu đầy đủ để mô hình học được các biểu diễn tốt. Goodfellow và cộng sự (2016) đã thảo luận trong cuốn "Deep Learning" rằng mạng nơ-ron học sâu thường cần lượng dữ liệu lớn để đạt hiệu suất tối ưu **goodfellow2016deep**.

Class-Weighted Loss là kỹ thuật có ảnh hưởng lớn nhất đến hiệu suất trên các lớp thiểu số. Trước khi áp dụng trọng số lớp, mô hình gần như bỏ qua hoàn toàn các lớp như *threat* và *identity_hate* do số lượng mẫu quá ít. Sau khi áp dụng trọng số lớp với clipping ở mức 50, mô hình bắt đầu nhận diện được các mẫu từ các lớp này, với recall dao động từ 0.68 đến 0.96 tùy mô hình. Lin và cộng sự (2017) đã nghiên cứu về Focal Loss như một phương pháp hiệu quả để xử lý dữ liệu mất cân bằng trong object detection và nhận thấy rằng việc giảm trọng số của các mẫu dễ phân loại giúp mô hình tập trung vào các mẫu khó **lin2017focal**. Tuy nhiên, precision vẫn thấp ở các lớp thiểu số (0.11-0.37), cho thấy vẫn còn nhiều dương tính giả và cần các cải tiến thêm như tăng cường dữ liệu hoặc các embedding đã được huấn luyện trước.

AdamW với weight decay giúp ngăn quá khớp và cải thiện khả năng tổng quát hóa của mô hình. Loshchilov và Hutter (2017) đã chứng minh rằng AdamW cải thiện đáng kể so với Adam thông thường trong việc regularization và đạt kết quả tốt hơn trên các bộ đánh giá **loshchilov2017adamw**. Gradient clipping đảm bảo quá trình huấn luyện ổn định, đặc biệt quan trọng đối với các mô hình RNN. Graves và cộng sự (2012) đã áp dụng gradient clipping thành công trong các mạng RNN cho nhận dạng giọng nói và nhận thấy rằng kỹ thuật này đặc biệt quan trọng khi huấn luyện trên các chuỗi dài **graves2012supervised**. Early Stopping đã ngăn chặn quá khớp hiệu quả, với hầu hết các mô hình dừng ở epoch 4-7 thay vì chạy đủ 10 epoch, cho thấy mô hình đã đạt đến điểm tối ưu trước khi bắt đầu quá khớp.

Warmup Cosine LR Scheduler giúp quá trình huấn luyện ổn định và hội tụ tốt hơn. Khi quan sát đường cong tốc độ học, ta thấy tốc độ học tăng tuyến tính trong giai đoạn warmup (khoảng 200 bước đầu), sau đó giảm dần theo hàm cosine trong phần còn lại. Vaswani và cộng sự (2017) đã sử dụng bộ lập lịch warmup trong huấn luyện Transformer và nhận thấy rằng nó giúp ổn định quá trình huấn luyện trong giai đoạn đầu **vaswani2017transformer**. Điều này cho phép mô hình khám phá rộng rãi không gian tham số ở giai đoạn đầu và tinh chỉnh ở giai đoạn cuối.

Kỹ thuật tìm kiếm các ngưỡng tối ưu riêng biệt cho từng lớp cũng mang lại cải thiện đáng kể. Các ngưỡng tối ưu thường cao hơn 0.5 đáng kể (từ 0.62 đến 0.89), cho thấy mô hình có xu hướng đưa ra xác suất khá cao cho cả các mẫu không

độc hại. Việc tăng ngưỡng giúp lọc bớt các dương tính giả và cải thiện precision. Fawcett (2006) đã thảo luận chi tiết về tầm quan trọng của việc lựa chọn ngưỡng phù hợp trong các bài toán phân loại và cách nó ảnh hưởng đến đường cong ROC và các độ đo hiệu suất khác nhau **fawcett2006introduction**.

0.6 Nhận xét chung

Qua quá trình thử nghiệm và đánh giá 7 mô hình học sâu khác nhau trên bài toán phân loại bình luận độc hại, một số nhận xét quan trọng có thể được rút ra. Thứ nhất, cơ chế Attention thực sự mang lại lợi ích đáng kể trong bài toán này. Attention BiLSTM vượt trội hơn BiLSTM ở cả ROC-AUC lẫn F1 Macro, và sự cải thiện này đến từ khả năng tập trung vào các từ quan trọng của attention thay vì tính bình quân tất cả các bước thời gian như trong mean pooling truyền thống. Kết quả này nhất quán với các nghiên cứu trước đây của Bahdanau và cộng sự (2014) về hiệu quả của attention trong các bài toán NLP **bahdanau2014attention**.

Thứ hai, kiến trúc hai chiều vượt trội hơn kiến trúc một chiều trong hầu hết các trường hợp. BiLSTM đạt kết quả tốt hơn LSTM, và Attention BiLSTM đạt kết quả tốt hơn Attention LSTM. Điều này cho thấy việc nắm bắt ngữ cảnh cả trước và sau một từ là quan trọng trong việc hiểu ý nghĩa của văn bản, đặc biệt khi có các từ phủ định hoặc các mẫu phức tạp. Graves và cộng sự (2012) đã chứng minh rằng việc xử lý hai chiều mang lại cải thiện đáng kể trong nhận dạng giọng nói **graves2012supervised**.

Thứ ba, việc xử lý dữ liệu mất cân bằng là yếu tố then chốt quyết định thành công của bài toán. Không có Class-Weighted Loss, mô hình gần như không thể nhận diện được các lớp thiểu số như threat (chỉ 0.2% dữ liệu). Với Class-Weighted Loss và clipping, recall cho tất cả các lớp đều đạt mức cao (0.68-0.97), cho thấy mô hình đã học được cách nhận diện các mẫu từ các lớp thiểu số. Lin và cộng sự (2017) đã nghiên cứu về Focal Loss như một phương pháp hiệu quả để xử lý dữ liệu mất cân bằng và nhận thấy rằng giảm trọng số của các mẫu dễ phân loại mang lại cải thiện đáng kể **lin2017focal**.

Thứ tư, Precision vẫn là điểm yếu chung của tất cả các mô hình, đặc biệt trên các lớp thiểu số. Precision thấp nhất là ở threat (0.09-0.37) và identity_hate (0.11-0.32), cho thấy mô hình vẫn tạo ra nhiều dương tính giả trên các lớp này. Nguyên nhân có thể là do số lượng mẫu quá ít khiến mô hình không thể học được các biểu diễn đủ tốt để phân biệt chính xác giữa các trường hợp thực sự và giả của các lớp thiểu số. Sokolova và Lapalme (2009) đã phân tích chi tiết về các độ đo hiệu suất cho bài toán phân loại và nhận thấy rằng F1-score là một độ đo cân bằng, nhưng precision và recall có thể thay đổi đáng kể tùy thuộc vào ngưỡng được

chọn sokolova2009systematic.

Thứ năm, Transformer với cấu hình hiện tại chưa đạt hiệu suất tốt như các mô hình RNN trong thử nghiệm này. Tuy nhiên, điều này không có nghĩa là Transformer không phù hợp cho bài toán này, mà có thể cần cấu hình khác (nhiều tầng hơn, các embedding đã được huấn luyện trước, nhiều epoch huấn luyện) để phát huy hết tiềm năng của kiến trúc. Vaswani và cộng sự (2017) đã huấn luyện Transformer với nhiều tầng hơn và đạt hiệu suất vượt trội trên bài toán dịch máy vaswani2017transformer.

Cuối cùng, sự kết hợp giữa RCNN (hai chiều + embedding fusion + max pooling) và Attention BiLSTM (hai chiều + attention) cho thấy hai hướng tiếp cận khác nhau để cải thiện biểu diễn: RCNN tập trung vào việc trích xuất các đặc trưng mạnh nhất (max pooling), trong khi Attention BiLSTM tập trung vào việc tổng hợp có trọng số các đặc trưng. Lai và cộng sự (2015) đã đề xuất kiến trúc RCNN và nhận thấy rằng việc kết hợp RNN và CNN mang lại hiệu quả tốt trên bài toán phân loại văn bản lai2015rcnn. Cả hai đều vượt trội hơn các mô hình cơ bản không có các cải tiến này.

0.7 Kết quả sau tối ưu hóa và đánh giá chéo

0.7.1 Kết quả trên Jigsaw sau tối ưu hóa

Ở lần chạy đầu (Bảng 1), hai mô hình dựa trên RNN dẫn đầu: RCNN đạt Best AUC cao nhất (0.9839) nhưng F1 Macro chỉ 0.5689, trong khi Attention BiLSTM đạt F1 Macro tốt nhất (0.6311) với AUC 0.9834. Transformer tụt lại với F1 Macro 0.4625, thấp hơn RCNN 0.1064 điểm.

Sau khi áp dụng các kỹ thuật tối ưu hóa, cả ba mô hình đều cải thiện đáng kể về F1 Macro (Bảng 2), trong khi AUC thay đổi không đáng kể (± 0.0007). Mức cải thiện cụ thể:

- **RCNN:** F1 Macro tăng từ 0.5689 lên 0.6530 (+14.8%), nhiều nhất về giá trị tuyệt đối nhờ masked pooling và label smoothing cải thiện hiệu chỉnh.
- **Transformer:** F1 Macro tăng từ 0.4625 lên 0.6274 (+35.7%), ấn tượng nhất về tỷ lệ, nhờ Pre-LN, CLS token và 4 tầng mã hóa — thu hẹp khoảng cách với RCNN từ 0.1064 xuống còn 0.0256.
- **Attention BiLSTM:** F1 Macro tăng nhẹ từ 0.6311 lên 0.6344 (+0.5%), do kiến trúc gốc vốn đã hoạt động tốt.

AUC gần như không thay đổi giữa các phiên bản, cho thấy khả năng xếp hạng các mẫu vốn đã tốt từ phiên bản gốc. Các kỹ thuật tối ưu hóa chủ yếu tác động đến việc cải thiện precision và recall tại các ngưỡng quyết định cụ thể, từ đó nâng cao

F1 Macro.

Mô hình	Phiên bản	AUC	F1 Macro	F1 Micro
RCNN	Gốc	0.9839	0.5689	0.6655
RCNN	Tối ưu	0.9846	0.6530	0.7703
Attention BiLSTM	Gốc	0.9834	0.6311	0.7324
Attention BiLSTM	Tối ưu	0.9833	0.6344	0.7608
Transformer	Gốc	0.9789	0.4625	0.5947
Transformer	Tối ưu	0.9797	0.6274	0.7497

Bảng 2: So sánh kết quả trên Jigsaw trước và sau tối ưu hóa

0.7.2 Kết quả trên HateXplain

Cả ba mô hình đều được huấn luyện trên HateXplain với cấu hình tối ưu. Khác với Jigsaw (đa nhãn, 6 nhãn), HateXplain là bài toán đa lớp với 3 nhãn nên sử dụng CrossEntropyLoss. Kết quả được trình bày trong Bảng 3.

Mô hình	AUC	F1 Macro	F1 Micro	Precision	Recall
RCNN	0.8156	0.6399	0.6571	0.6484	0.6414
Attention BiLSTM	0.8039	0.6169	0.6380	0.6239	0.6229
Transformer	0.8087	0.6375	0.6402	0.6415	0.6356

Bảng 3: Kết quả trên HateXplain — 3 mô hình

Nhìn chung, AUC trên HateXplain thấp hơn đáng kể so với Jigsaw (0.80-0.82 so với 0.98), phản ánh độ khó cao hơn của dữ liệu từ Twitter và Reddit cũng như số lượng mẫu ít hơn (20,109 so với 159,571). RCNN đạt AUC cao nhất (0.8156) và F1 Macro cao nhất (0.6399), khẳng định lại ưu thế của kiến trúc kết hợp BiLSTM với max pooling. Transformer đạt F1 Macro 0.6375, rất sát với RCNN (0.6399) và vượt trội so với Attention BiLSTM (0.6169). Đây là điểm khác biệt quan trọng so với kết quả trên Jigsaw, nơi Transformer ở phiên bản gốc có F1 Macro thấp hơn nhiều so với các mô hình còn lại.

0.7.3 So sánh trước và sau tối ưu hóa

Bảng 4 tổng hợp mức cải thiện của từng mô hình trên cả hai bộ dữ liệu.

Mô hình	Jigsaw (gốc)	Jigsaw (tối ưu)	HateXplain
RCNN	0.5689	0.6530 (+0.0841)	0.6399
Attention BiLSTM	0.6311	0.6344 (+0.0033)	0.6169
Transformer	0.4625	0.6274 (+0.1649)	0.6375

Bảng 4: Tổng hợp mức cải thiện F1 Macro

0.7.4 Thảo luận về hiệu suất của Transformer

Kết quả từ các thử nghiệm tối ưu hóa và đánh giá chéo cung cấp bằng chứng quan trọng để đánh giá giả thuyết rằng Transformer kém hơn RCNN và Attention BiLSTM do phụ thuộc vào bài toán.

Trên Jigsaw (đa nhãn), Transformer phiên bản gốc (Post-LN, mean pooling, 2 tầng) có F1 Macro 0.4625 - thấp hơn RCNN (0.5689) khoảng 0.1064 điểm. Tuy nhiên, sau khi áp dụng Pre-LN, CLS token và 4 tầng mã hóa, Transformer đạt F1 Macro 0.6274 - thu hẹp khoảng cách với RCNN (0.6530) xuống chỉ còn 0.0256 điểm (Bảng 2). Mức cải thiện 35.7% cho thấy vấn đề chính không phải do kiến trúc Transformer yếu kém, mà do cấu hình chưa phù hợp.

Trên HateXplain (đa lớp), Transformer đạt F1 Macro 0.6375, gần như tương đương với RCNN (0.6399) và vượt trội so với Attention BiLSTM (0.6169). Kết quả này càng củng cố nhận định rằng Transformer hoàn toàn có thể cạnh tranh với các mô hình dựa trên RNN khi được cấu hình đúng, đặc biệt trên bài toán đa lớp.

Hai yếu tố then chốt giúp cải thiện Transformer là:

- **Pre-LN:** LayerNorm được đặt trước sublayer thay vì sau residual connection, giúp gradient flow ổn định trong backpropagation, tránh gradient explosion ở các tầng gần đầu vào. Xiong và cộng sự (2020) **xiong2020layer** đã chứng minh Pre-LN vượt trội so với Post-LN, đặc biệt với Transformer nhiều tầng.
- **CLS token:** Token đặc biệt học cách tổng hợp toàn bộ câu thông qua cơ chế self-attention, thay vì mean pooling làm pha loãng tín hiệu từ các từ quan trọng. Phương pháp này được sử dụng phổ biến trong BERT **devlin2019bert**.

Tuy nhiên, RCNN vẫn là mô hình có AUC cao nhất trên cả hai bộ dữ liệu (0.9846 trên Jigsaw, 0.8156 trên HateXplain), cho thấy kiến trúc kết hợp BiLSTM và max pooling là lựa chọn ổn định và đáng tin cậy cho bài toán phân loại văn bản độc hại ở nhiều điều kiện dữ liệu.